PROVISIONAL PATENT APPLICATION

IN THE UNITED STATES PATENT AND TRADEMARK OFFICE

TITLE OF INVENTION

BLOCKCHAIN-BASED HOMEOWNERS ASSOCIATION GOVERNANCE SYSTEM WITH SOULBOUND PROPERTY TOKENS, TIERED CATEGORY-SPECIFIC QUORUM VOTING, AUTOMATED TREASURY ALLOCATION, IMMUTABLE DOCUMENT REGISTRY, AND COMPOSITE COMMUNITY HEALTH SCORING

CROSS-REFERENCE TO RELATED APPLICATIONS

This application does not claim priority to any prior application. This is an original provisional patent application.

FIELD OF THE INVENTION

The present invention relates generally to decentralized governance systems, and more particularly to a blockchain-based platform for homeowners association (HOA) management that employs soulbound non-fungible tokens (NFTs) as property ownership credentials and governance instruments, tiered category-specific quorum thresholds enforced via smart contracts, automated treasury allocation mechanisms, an immutable document registry integrated with permanent decentralized storage, and a composite algorithmic community health scoring system operating on a Layer 2 Ethereum-compatible blockchain network.

BACKGROUND OF THE INVENTION

Homeowners associations in the United States collectively manage over 358,000 communities representing more than 74 million residents and trillions of dollars in real estate value. Despite their scale and significance, traditional HOA governance systems suffer from systemic deficiencies that result in financial misconduct, record manipulation, disenfranchisement of homeowners, and lack of transparency.

Problem 1: Financial Mismanagement and Embezzlement. HOA financial fraud and embezzlement represent losses estimated to exceed $300 billion annually across the United States. Traditional HOA treasury management relies on paper checks, manual bookkeeping, and management company intermediaries who exercise unilateral control over association funds. Homeowners typically have no real-time visibility into dues collection rates, expenditure approvals, or fund balances. Reserve funds, intended for long-term capital expenditures such as roof replacement and pavement repair, are routinely co-mingled with or raided from operating accounts. Audits are periodic and expensive, and internal controls are often absent in smaller associations.

Problem 2: Document Manipulation and Record Alteration. HOA management companies and boards have been documented altering, destroying, or selectively disclosing governing documents including Covenants, Conditions and Restrictions (CC&Rs), meeting minutes, contracts, and financial statements. Homeowners disputing enforcement actions frequently discover that the documents produced by management differ from versions they previously received. No immutable, verifiable record of community documents exists in conventional HOA management systems. This creates an

environment of asymmetric information where management holds informational advantage over homeowners.

Problem 3: Governance Manipulation Through Low-Participation Quorum Exploitation. Traditional HOA bylaws commonly establish single flat quorum thresholds (typically 10-20% of members) for all proposal categories, regardless of the significance of the matter being voted upon. This design flaw enables boards and management to push through consequential changes — including CC&R amendments, special assessments, and board composition changes — with minimal homeowner participation, often by holding votes at inconvenient times or with inadequate notice. Constitutional amendments affecting fundamental property rights may pass with the same threshold as routine maintenance approvals.

Problem 4: Proxy Vote Accumulation and Governance Capture. In associations using traditional voting mechanisms, a single actor may accumulate unlimited proxy votes, enabling governance capture by well-resourced parties such as developers, management companies, or investor landlords. This violates the fundamental HOA principle of proportional democratic representation among property owners.

Problem 5: Lack of Objective Community Health Metrics. Prospective homebuyers, lenders, and current residents have no standardized, objective metric for evaluating the governance quality or financial health of an HOA community. Current tools require access to manually compiled financial documents, audits, and meeting minutes that may be incomplete, outdated, or deliberately obscured.

Problem 6: Inefficiencies in Dues Collection and Allocation. Manual dues collection processes are inefficient, error-prone, and susceptible to delayed allocation between operating and reserve funds. Automated payment processing with guaranteed, rules-based fund allocation does not exist in conventional HOA management systems.

Prior art in blockchain voting systems has addressed some aspects of governance transparency but has not provided an integrated, HOA-specific system that combines property ownership tokenization with governance rights, tiered category-specific quorum enforcement, automated treasury management, permanent document registration, and community health scoring within a single coherent platform deployed on a production blockchain network.

Accordingly, there exists a need in the art for a comprehensive blockchain-based HOA governance platform that addresses the foregoing deficiencies.

SUMMARY OF THE INVENTION

The present invention provides a blockchain-based homeowners association governance and management platform comprising five integrated inventive systems deployed on a Layer 2 Ethereum-compatible blockchain network.

In a first aspect, the invention provides a soulbound property NFT governance system wherein each residential lot within a governed community is represented by a unique ERC-721 non-fungible token ("Property NFT") that simultaneously serves as a proof-of-ownership credential and a governance voting instrument. The Property NFT is "soulbound" in that token transfers are restricted by role-based access controls requiring explicit approval from authorized governance actors, preventing unauthorized transfer or accumulation of voting rights. Each Property NFT carries exactly one (1) unit of voting weight, implementing a one-lot-one-vote governance model that prevents whale accumulation. Property metadata including lot number, street address, and parcel square footage is stored immutably on-chain within the token's data structures.

In a second aspect, the invention provides a tiered governance system with category-specific quorum thresholds and approval requirements. The system defines four (4) proposal categories — Routine, Financial, Governance, and Constitutional — each associated with distinct minimum quorum percentages and approval threshold percentages enforced at the smart contract level. Constitutional proposals require a supermajority quorum of sixty-seven percent (67%) and a supermajority approval threshold of sixty-six point seven percent (66.7%), while Routine proposals require a fifteen percent (15%) quorum with simple majority approval. A timelock mechanism delays execution of passed proposals, providing a review window and preventing flash governance attacks.

In a third aspect, the invention provides an automated treasury management system that receives HOA dues payments in a U.S. Dollar-pegged stablecoin (USDC), automatically allocates incoming payments between an operating fund and a reserve fund according to a configurable split ratio (defaulting to 80% operating / 20% reserve), and enforces that all expenditures from treasury funds be authorized through the governance proposal system. The system supports both quarterly and annual payment schedules with a discount for annual prepayment.

In a fourth aspect, the invention provides an immutable document registry wherein HOA governing documents are registered on-chain via a cryptographic content hash, with the full document content stored on a permanent, decentralized storage network (Arweave). The registry enables any party to verify whether a document file they possess matches the version registered on-chain by computing and comparing its hash. The registry supports document versioning and supersession relationships, maintaining a complete, tamper-evident audit trail of all community documents.

In a fifth aspect, the invention provides a composite community health scoring algorithm that computes a real-time numerical health score between zero (0) and one hundred (100) for a governed community by weighting and combining five on-chain metrics: dues collection rate, treasury health ratio, governance participation rate, document compliance rate, and community size indicator.

BRIEF DESCRIPTION OF THE DRAWINGS

The accompanying drawings, which are incorporated in and constitute a part of this specification, illustrate preferred embodiments of the invention.

FIG. 1 is a system architecture diagram illustrating the five primary smart contract components (PropertyNFT, FaircroftGovernor, FaircroftTreasury, DocumentRegistry, and HealthScorer), their relationships to one another, their integration with the Base Layer 2 blockchain network, and the Arweave permanent storage network.

FIG. 2 is a data flow diagram illustrating the lifecycle of a Property NFT from initial minting by an authorized administrator through metadata storage, voting power delegation, governance participation, and the restricted transfer process requiring board approval.

FIG. 3 is a state machine diagram illustrating the lifecycle of a governance proposal through states: Pending, Active, Canceled, Defeated, Succeeded, Queued, Executed, and Expired, with transitions annotated by category-specific quorum and threshold conditions.

FIG. 4 is a data flow diagram illustrating the automated treasury allocation process: USDC payment receipt, the 80/20 operating/reserve fund split computation, fund segregation into separate accounting balances within the treasury contract, and the governance-gated expenditure withdrawal flow.

FIG. 5 is a diagram illustrating the document registration and verification workflow: document upload, SHA-256 hash computation, on-chain registration transaction, Arweave content upload, and the drag-

and-drop client-side verification process that computes and compares a file hash against the on-chain record.

FIG. 6 is a scoring algorithm diagram showing the five weighted input metrics, their individual calculation formulas, the application of weighting coefficients, and the summation producing the composite community health score.

FIG. 7 is a block diagram of the frontend application architecture illustrating Next.js server and client component boundaries, wagmi/viem blockchain interaction hooks, RainbowKit wallet connection interface, and the data fetching patterns for on-chain state.

DETAILED DESCRIPTION OF PREFERRED EMBODIMENTS

The following detailed description sets forth specific embodiments of the invention. Those skilled in the art will recognize that the invention may be embodied in other forms without departing from the spirit of the disclosure.

I. SYSTEM OVERVIEW

The inventive system, referred to herein as the "SuvrenHOA Platform" or the "Platform," comprises a collection of Solidity smart contracts deployed on the Base blockchain network (a Layer 2 Ethereum-compatible network operated by Coinbase, Inc.) and a web frontend application. The smart contracts are written in Solidity version 0.8.24 and compiled with optimizer settings enabled. The frontend is built with Next.js version 16, React version 19, and utilizes the wagmi library (version 2.x) for Ethereum state management, the viem library for low-level contract interactions, and the RainbowKit library for wallet connection interfaces.

The five primary smart contracts of the system are:

1. **PropertyNFT** — ERC-721 non-fungible token contract implementing the soulbound property ownership and governance credential system.
2. **FaircroftGovernor** — OpenZeppelin Governor-derived contract implementing the tiered category-specific quorum governance system.
3. **FaircroftTreasury** — USDC payment processing and automated fund allocation contract.
4. **DocumentRegistry** — On-chain document hash registry with Arweave integration.
5. **HealthScorer** — Composite community health score computation contract (or off-chain computation reading on-chain state).

II. INVENTION 1: SOULBOUND PROPERTY NFT GOVERNANCE SYSTEM

A. Contract Architecture

The PropertyNFT contract inherits from the following OpenZeppelin base contracts: `ERC721`, `ERC721Enumerable`, `ERC721URIStorage`, `EIP712`, `ERC721Votes`, and `AccessControl`. The combined inheritance provides ERC-721 token functionality, enumerable token indexing, per-token URI metadata storage, EIP-712 typed structured data hashing for permit-style signatures, and the ERC-5805 Votes extension that enables checkpoint-based vote delegation compatible with the OpenZeppelin Governor framework.

The contract defines the following roles using the `AccessControl` role-based permission system:

```
bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
bytes32 public constant TRANSFER_APPROVER_ROLE =
```

```
keccak256("TRANSFER_APPROVER_ROLE");
bytes32 public constant METADATA_UPDATER_ROLE = keccak256("METADATA_UPDATER_ROLE");
```

`DEFAULT_ADMIN_ROLE` (inherited from `AccessControl`) is assigned to the deploying governance multisig address.

B. Property Metadata Storage

Each property is associated with a `PropertyData` struct stored in a contract-level mapping:

```
struct PropertyData {
    uint256 lotNumber;
    string streetAddress;
    uint256 squareFootage;
    bool isActive;
    uint256 mintTimestamp;
}

mapping(uint256 tokenId => PropertyData) public propertyData;
```

Property metadata is redundantly stored both in the `PropertyData` struct (for on-chain query efficiency) and encoded in a JSON metadata document conforming to the ERC-721 metadata standard, with the `tokenURI` pointing to an Arweave-hosted immutable JSON file. The `tokenURI` value is set at mint time and may be updated by `METADATA_UPDATER_ROLE` to reflect property data corrections, with update events emitted for auditability.

C. Soulbound Transfer Restriction

The soulbound property of Property NFTs is implemented by overriding the `_update` function (the internal transfer hook in OpenZeppelin ERC721 v5.x), which is invoked on all token transfers, mints, and burns:

```
function _update(
    address to,
    uint256 tokenId,
    address auth
) internal override(ERC721, ERC721Enumerable, ERC721Votes) returns (address) {
    address from = _ownerOf(tokenId);

    // Allow mints (from == address(0)) and burns (to == address(0))
    // For transfers between non-zero addresses, require TRANSFER_APPROVER_ROLE
    if (from != address(0) && to != address(0)) {
        if (!hasRole(TRANSFER_APPROVER_ROLE, msg.sender)) {
            revert TransferNotApproved(tokenId, from, to);
        }
    }

    return super._update(to, tokenId, auth);
}
```

This override ensures that Property NFTs cannot be transferred on secondary markets, cannot be used as collateral (preventing forced liquidation transfers), and cannot be accumulated by any actor without

explicit board approval. The only actors that may call the transfer function with effect are those holding the `TRANSFER_APPROVER_ROLE`, which is managed through the governance process itself, creating a self-reinforcing governance loop.

D. One-Lot-One-Vote Enforcement

The `ERC721Votes` base contract implements vote weight as the count of tokens held by an address. Because Property NFTs are soulbound, each property owner address holds exactly one (1) Property NFT (one lot), resulting in exactly one (1) unit of voting weight per household. This is further enforced by the minting function:

```
function mintProperty(
    address to,
    uint256 lotNumber,
    string calldata streetAddress,
    uint256 squareFootage,
    string calldata tokenURI_
) external onlyRole(MINTER_ROLE) returns (uint256) {
    require(balanceOf(to) == 0, "Address already holds a Property NFT");

    uint256 tokenId = _nextTokenId++;
    _safeMint(to, tokenId);
    _setTokenURI(tokenId, tokenURI_);

    propertyData[tokenId] = PropertyData({
        lotNumber: lotNumber,
        streetAddress: streetAddress,
        squareFootage: squareFootage,
        isActive: true,
        mintTimestamp: block.timestamp
    });

    emit PropertyMinted(tokenId, to, lotNumber, streetAddress);
    return tokenId;
}
```

The `require(balanceOf(to) == 0)` check prevents minting a second Property NFT to any address that already holds one, enforcing the one-lot-one-vote invariant at the smart contract level.

E. Vote Delegation

Voting power under the `ERC721Votes` extension must be actively delegated to be counted in governance proposals. A property owner delegates their voting weight by calling `delegate(address delegatee)` or `delegateBySig(...)` for gasless delegation. Owners may delegate to themselves (the typical case) or to another address (proxy voting). Because the system is soulbound, delegated voting power cannot exceed one (1) per property, preventing the accumulation of voting power through token purchase.

III. INVENTION 2: TIERED GOVERNANCE WITH CATEGORY-SPECIFIC QUORUMS

A. Proposal Category System

The `FaircroftGovernor` contract extends OpenZeppelin's `Governor`, `GovernorSettings`, `GovernorCountingSimple`, `GovernorVotes`, `GovernorVotesQuorumFraction`, and `GovernorTimelockControl` base contracts, with custom overrides implementing the category-specific quorum system.

The four proposal categories are defined as an enumeration:

```
enum ProposalCategory {
    Routine,       // 0: Maintenance, minor decisions
    Financial,     // 1: Budget items, expenditures
    Governance,    // 2: Rule changes, board composition
    Constitutional // 3: CC&R amendments, fundamental changes
}
```

Each category is associated with a `CategoryConfig` struct:

```
struct CategoryConfig {
    uint256 quorumNumerator;      // Quorum as percentage numerator (basis points
/100)
    uint256 approvalThreshold;    // Required approval percentage (basis points
/100)
    uint256 timelockDelay;        // Seconds between passage and execution
    string description;
}
```

The default category configurations at deployment are:

Category	Quorum Numerator	Approval Threshold	Timelock Delay
Routine	15	51	1 day
Financial	33	51	2 days
Governance	51	51	3 days
Constitutional	67	67	7 days

A mapping associates each proposal ID with its category at creation time:

```
mapping(uint256 proposalId => ProposalCategory) public proposalCategories;
```

B. Proposal Creation with Category Assignment

The `propose` function is overridden to accept an additional `ProposalCategory` parameter:

```
function proposeWithCategory(
    address[] memory targets,
    uint256[] memory values,
    bytes[] memory calldatas,
    string memory description,
    ProposalCategory category
```

```
) public returns (uint256) {
    uint256 proposalId = super.propose(targets, values, calldatas, description);
    proposalCategories[proposalId] = category;
    emit ProposalCategorySet(proposalId, category);
    return proposalId;
}
```

C. Category-Specific Quorum Override

The `quorum(uint256 blockNumber)` function is overridden to return the category-appropriate quorum for each proposal:

```
function quorum(uint256 proposalId) public view returns (uint256) {
    ProposalCategory category = proposalCategories[proposalId];
    CategoryConfig memory config = categoryConfigs[category];
    uint256 totalSupply = token().getPastTotalSupply(proposalSnapshot(proposalId));
    return (totalSupply * config.quorumNumerator) / 100;
}
```

This override replaces the flat quorum fraction used by `GovernorVotesQuorumFraction` with a per-proposal lookup that enforces the category-appropriate minimum participation threshold.

D. Category-Specific Approval Threshold Override

The `_voteSucceeded` function is overridden to enforce category-specific approval thresholds:

```
function _voteSucceeded(uint256 proposalId) internal view override returns (bool) {
    ProposalCategory category = proposalCategories[proposalId];
    CategoryConfig memory config = categoryConfigs[category];

    (uint256 againstVotes, uint256 forVotes, ) = proposalVotes(proposalId);
    uint256 totalVotes = forVotes + againstVotes;

    if (totalVotes == 0) return false;

    // Check approval threshold (forVotes / totalVotes >= approvalThreshold / 100)
    return (forVotes * 100) >= (totalVotes * config.approvalThreshold);
}
```

E. Quorum Satisfaction Check Override

The `_quorumReached` function is overridden to use the proposal-category-specific quorum:

```
function _quorumReached(uint256 proposalId) internal view override returns (bool) {
    (, uint256 forVotes, uint256 abstainVotes) = proposalVotes(proposalId);
    return quorum(proposalId) <= forVotes + abstainVotes;
}
```

F. Timelock Integration

The `FaircroftGovernor` integrates with OpenZeppelin's `TimelockController` contract. Upon proposal passage, the governor queues the proposal's call data with a minimum delay corresponding

to the category's `timelockDelay`. The timelock delay provides a window during which homeowners can observe passed proposals before execution and, if governance rules allow, contest or cancel execution. For Constitutional proposals, a seven-day delay is enforced, providing substantial review time for changes to foundational community rules.

IV. INVENTION 3: AUTOMATED TREASURY MANAGEMENT WITH SPLIT ALLOCATION

A. Contract Architecture

The `FaircroftTreasury` contract manages all HOA financial flows. It interacts with the USDC ERC-20 token contract (USDC deployed on Base at a known address) and enforces the automated allocation split, payment schedule logic, and governance-gated expenditure system.

B. Fund Segregation

The treasury maintains separate internal accounting balances for two funds:

```
uint256 public operatingFundBalance;
uint256 public reserveFundBalance;
```

These balances are internal accounting variables maintained within a single contract that holds the actual USDC token balance, providing logical fund segregation without requiring separate contract deployments or token transfers between accounts.

C. Automated 80/20 Allocation

The `payDues` function processes incoming USDC payments and automatically allocates them between funds:

```
uint256 public constant OPERATING_SPLIT_BPS = 8000;  // 80.00%
uint256 public constant RESERVE_SPLIT_BPS = 2000;     // 20.00%
uint256 public constant BPS_DENOMINATOR = 10000;

function payDues(bool isAnnual) external nonReentrant {
    uint256 amount;

    if (isAnnual) {
        amount = ANNUAL_DUES_AMOUNT;  // 5% discount applied at constant definition
    } else {
        amount = QUARTERLY_DUES_AMOUNT;
    }

    // Transfer USDC from payer to treasury
    usdc.safeTransferFrom(msg.sender, address(this), amount);

    // Compute split
    uint256 operatingPortion = (amount * OPERATING_SPLIT_BPS) / BPS_DENOMINATOR;
    uint256 reservePortion = amount - operatingPortion;  // Remainder prevents
rounding loss

    // Update internal accounting
    operatingFundBalance += operatingPortion;
```

```
    reserveFundBalance += reservePortion;

    // Record payment
    paymentRecords[msg.sender].push(PaymentRecord({
        amount: amount,
        timestamp: block.timestamp,
        isAnnual: isAnnual,
        periodCovered: _currentPeriod()
    }));

    emit DuesPaid(msg.sender, amount, operatingPortion, reservePortion, isAnnual);
}
```

The use of basis points (BPS) arithmetic ensures integer arithmetic precision. The reserve portion is computed as `amount - operatingPortion` rather than `(amount * RESERVE_SPLIT_BPS) / BPS_DENOMINATOR` to prevent any rounding discrepancy from resulting in unaccounted USDC within the contract.

D. Payment Schedule and Discount

Annual dues are set at a five percent (5%) discount relative to four quarterly payments:

```
uint256 public constant QUARTERLY_DUES_AMOUNT = 200e6;    // $200.00 USDC (6
decimals)
uint256 public constant ANNUAL_DUES_AMOUNT = 760e6;       // $760.00 USDC (5%
discount)
// 4 × $200 = $800; 5% discount: $800 × 0.95 = $760
```

The contract tracks payment status per property owner address, enabling querying of delinquency status for both health scoring and enforcement purposes.

E. Governance-Gated Expenditures

All expenditures from treasury funds require an executed governance proposal. The `executeExpenditure` function is callable only by the `TimelockController` address (i.e., only upon successful governance execution):

```
function executeExpenditure(
    address recipient,
    uint256 amount,
    FundType fundType,
    string calldata description
) external onlyRole(EXECUTOR_ROLE) nonReentrant {
    if (fundType == FundType.Operating) {
        require(operatingFundBalance >= amount, "Insufficient operating funds");
        operatingFundBalance -= amount;
    } else {
        require(reserveFundBalance >= amount, "Insufficient reserve funds");
        reserveFundBalance -= amount;
    }

    usdc.safeTransfer(recipient, amount);
```

```
    emit ExpenditureExecuted(recipient, amount, fundType, description,
block.timestamp);
}
```

The `EXECUTOR_ROLE` is held exclusively by the `TimelockController` contract address, ensuring that no human actor can directly withdraw funds regardless of their role level. Every expenditure generates an immutable on-chain event log entry.

V. INVENTION 4: IMMUTABLE DOCUMENT REGISTRY WITH ARWEAVE INTEGRATION

A. Document Data Structures

The `DocumentRegistry` contract stores document registration records using the following data structures:

```
struct DocumentRecord {
    bytes32 contentHash;          // SHA-256 hash of document content
    string arweaveTransactionId;  // Arweave TxID where full document is stored
    string documentTitle;
    DocumentCategory category;
    uint256 registrationTimestamp;
    address registeredBy;
    uint256 supersededBy;         // Token ID of superseding document (0 if current)
    uint256 supersedes;           // Token ID of document this one supersedes (0 if
original)
    bool isActive;
}

mapping(uint256 documentId => DocumentRecord) public documents;
uint256 private _nextDocumentId;
```

The `DocumentCategory` enumeration classifies documents:

```
enum DocumentCategory {
    CCR,              // Covenants, Conditions & Restrictions
    Bylaws,
    MeetingMinutes,
    Budget,
    AnnualReport,
    Contract,
    Amendment,
    Resolution,
    Other
}
```

B. Document Registration

Documents are registered by authorized roles (board members, management) by submitting a content hash and Arweave transaction ID:

```
function registerDocument(
    bytes32 contentHash,
    string calldata arweaveTxId,
    string calldata title,
    DocumentCategory category,
    uint256 supersedes
) external onlyRole(REGISTRAR_ROLE) returns (uint256) {
    require(contentHash != bytes32(0), "Invalid content hash");
    require(bytes(arweaveTxId).length > 0, "Arweave TxID required");

    uint256 documentId = _nextDocumentId++;

    documents[documentId] = DocumentRecord({
        contentHash: contentHash,
        arweaveTransactionId: arweaveTxId,
        documentTitle: title,
        category: category,
        registrationTimestamp: block.timestamp,
        registeredBy: msg.sender,
        supersededBy: 0,
        supersedes: supersedes,
        isActive: true
    });

    // If this supersedes a prior document, mark it as superseded
    if (supersedes != 0) {
        require(documents[supersedes].isActive, "Prior document not active");
        documents[supersedes].supersededBy = documentId;
        documents[supersedes].isActive = false;
        emit DocumentSuperseded(supersedes, documentId);
    }

    emit DocumentRegistered(documentId, contentHash, arweaveTxId, category,
msg.sender);
    return documentId;
}
```

The content hash is computed client-side using the SubtleCrypto Web API (`crypto.subtle.digest('SHA-256', fileBuffer)`) before submission, ensuring the hash matches what a third-party verifier would independently compute from the document file.

C. Document Verification

Any party may verify a document by calling:

```
function verifyDocument(
    uint256 documentId,
    bytes32 claimedHash
) external view returns (bool matches, DocumentRecord memory record) {
    record = documents[documentId];
```

```
    matches = (record.contentHash == claimedHash);
}
```

The frontend implements a drag-and-drop verification interface: a user drags a document file onto the verification widget, the browser computes the SHA-256 hash client-side (no server upload required), and the application queries the contract to determine whether the hash matches any registered document record. This enables any homeowner to instantly verify whether a document they hold matches the version registered on-chain, without requiring technical blockchain knowledge.

D. Arweave Permanent Storage Integration

Documents are stored on the Arweave network, a blockchain-based permanent storage protocol that guarantees data persistence through an endowment-based economic model. The Arweave transaction ID stored on-chain enables any user to retrieve the full document from the Arweave gateway network (`https://arweave.net/{txId}`). Because the document content hash is registered on-chain simultaneously, the combination of (Arweave TxID → document content) + (on-chain contentHash → verified hash) provides a two-layer verifiability guarantee: the document retrieved from Arweave can be independently verified against the immutable on-chain hash.

VI. INVENTION 5: COMPOSITE COMMUNITY HEALTH SCORING ALGORITHM

A. Algorithm Overview

The Community Health Scoring system computes a normalized score in the range [0, 100] representing the overall governance quality and financial health of a governed HOA community. The score is computed from five weighted component metrics, each derived entirely from on-chain state, enabling trustless, real-time computation without dependence on off-chain oracles or manual data entry.

B. Component Metrics and Weighting

The five components and their weights are:

Component	Weight	Max Points
Dues Collection Rate	0.30	30
Treasury Health	0.25	25
Governance Participation	0.20	20
Document Compliance	0.15	15
Community Size	0.10	10

C. Component Calculation Formulas

Dues Collection Rate (max 30 points):

```
duesScore = (activePayingMembers / totalActiveMembers) × 30
```

Where `activePayingMembers` is the count of Property NFT holders whose payment records indicate a payment within the current or immediately prior period, and `totalActiveMembers` is the count of active (non-burned) Property NFTs.

Treasury Health (max 25 points):

```
reserveRatio = reserveFundBalance / (reserveFundBalance + operatingFundBalance)
treasuryScore = min(reserveRatio / targetReserveRatio, 1.0) × 25
```

Where `targetReserveRatio` is a governance-configurable parameter (default: 0.20, representing the 20% reserve allocation target).

Governance Participation Rate (max 20 points):

```
recentProposals = proposals voted on in last 90 days
avgParticipation = mean(participationRate(p) for p in recentProposals)
participationScore = avgParticipation × 20
```

Where `participationRate(p) = (forVotes(p) + againstVotes(p) + abstainVotes(p)) / totalSupplyAtProposalSnapshot(p)`.

Document Compliance Rate (max 15 points):

```
requiredDocTypes = [CCR, Bylaws, MeetingMinutes, Budget]
presentDocTypes = count(requiredDocTypes where activeDocument exists)
documentScore = (presentDocTypes / len(requiredDocTypes)) × 15
```

The system checks whether at least one active document of each required category exists in the DocumentRegistry.

Community Size (max 10 points):

```
memberCount = totalSupply() of PropertyNFT
sizeScore = min(memberCount / scalingTarget, 1.0) × 10
```

Where `scalingTarget` is a configurable constant (default: 50 properties), reflecting that larger communities with more voting participants have more robust governance.

D. Composite Score Computation

```
function computeHealthScore(
    address propertyNFT,
    address treasury,
    address governor,
    address documentRegistry
) external view returns (uint256 score, ComponentScores memory components) {
    components.duesScore = _computeDuesScore(propertyNFT, treasury);
    components.treasuryScore = _computeTreasuryScore(treasury);
    components.participationScore = _computeParticipationScore(governor,
propertyNFT);
    components.documentScore = _computeDocumentScore(documentRegistry);
    components.sizeScore = _computeSizeScore(propertyNFT);

    score = components.duesScore
          + components.treasuryScore
          + components.participationScore
          + components.documentScore
```

```
          + components.sizeScore;

    // score is in range [0, 100] by construction
}
```

The composite score is bounded to [0, 100] by construction, as each component is bounded to its respective maximum and the maximum components sum to 100.

VII. FRONTEND APPLICATION ARCHITECTURE

The frontend application is built with Next.js version 16 utilizing the App Router architecture, implementing React Server Components for data fetching and static rendering and React Client Components for interactive blockchain interactions.

Blockchain state is accessed through wagmi hooks (`useReadContract` , `useWriteContract` , `useWaitForTransactionReceipt`) that wrap viem's contract read/write primitives. The RainbowKit library provides the wallet connection UI supporting MetaMask, Coinbase Wallet, WalletConnect, and other EIP-1193-compatible browser wallets.

The application implements thirty-one (31) or more distinct routes covering governance proposal creation and voting, property registration management, treasury dashboards, document registration and verification, and community health score display.

CLAIMS

Claim 1. A computer-implemented system for blockchain-based homeowners association governance, the system comprising: a non-fungible token (NFT) contract deployed on a blockchain network, the NFT contract configured to mint unique property tokens, each property token representing a distinct residential lot, wherein each property token carries exactly one unit of voting weight; a transfer restriction mechanism within the NFT contract that prevents transfer of property tokens without authorization from a designated role, rendering the property tokens soulbound; a governance contract deployed on the blockchain network, the governance contract configured to receive votes cast using voting weight derived from held and delegated property tokens and to tally votes to determine proposal outcomes; and a treasury contract deployed on the blockchain network, the treasury contract configured to receive due payments and automatically allocate received payments between an operating fund and a reserve fund according to a predefined split ratio.

Claim 2. The system of Claim 1, wherein the NFT contract further comprises on-chain property metadata storage including at least a lot number, a street address, and a parcel square footage associated with each property token.

Claim 3. The system of Claim 1, wherein the transfer restriction mechanism comprises an override of the ERC-721 token transfer internal function that reverts any transfer transaction not executed by an address holding a designated transfer approver role in an access control registry.

Claim 4. The system of Claim 1, wherein the governance contract is further configured to classify each governance proposal into one of a plurality of proposal categories, each proposal category associated with a distinct minimum quorum threshold and a distinct minimum approval threshold.

Claim 5. The system of Claim 4, wherein the plurality of proposal categories comprises at least: a routine category associated with a first quorum threshold of approximately fifteen percent (15%) of total voting supply; a financial category associated with a second quorum threshold of approximately thirty-three percent (33%) of total voting supply; a governance category associated with a third

quorum threshold of approximately fifty-one percent (51%) of total voting supply; and a constitutional category associated with a fourth quorum threshold of approximately sixty-seven percent (67%) of total voting supply.

Claim 6. The system of Claim 5, wherein the constitutional category is further associated with an approval threshold requiring that affirmative votes represent at least sixty-six point seven percent (66.7%) of all votes cast on a constitutional proposal.

Claim 7. The system of Claim 4, wherein the governance contract further comprises a timelock mechanism that enforces a minimum delay period between the passage of a governance proposal and the execution of the proposal's encoded on-chain actions, wherein the minimum delay period varies by proposal category.

Claim 8. The system of Claim 1, wherein the split ratio is eighty percent (80%) to the operating fund and twenty percent (20%) to the reserve fund, and wherein the operating and reserve fund balances are maintained as separate internal accounting variables within a single treasury contract that holds actual token balances.

Claim 9. The system of Claim 1, wherein the treasury contract is further configured to accept due payments in a U.S. Dollar-pegged stablecoin token, and wherein the treasury contract supports at least two payment schedule options comprising a quarterly payment option and an annual payment option, and wherein the annual payment option is associated with a discount relative to the aggregate of four quarterly payments.

Claim 10. The system of Claim 1, further comprising a document registry contract deployed on the blockchain network, the document registry contract configured to store a cryptographic content hash of a governance document on-chain and to store a reference to a location on a permanent decentralized storage network where the full document content is accessible.

Claim 11. The system of Claim 10, wherein the document registry contract is further configured to associate each registered document with a document category, and to maintain versioning relationships between documents through supersession references that identify a prior document superseded by a current document.

Claim 12. The system of Claim 10, wherein the permanent decentralized storage network is the Arweave network, and wherein the reference stored on-chain comprises an Arweave transaction identifier.

Claim 13. The system of Claim 10, further comprising a client-side document verification interface configured to: receive a document file provided by a user; compute a cryptographic hash of the document file entirely within a user's browser environment without uploading the document to a server; and compare the computed hash against a content hash stored in the document registry contract to determine whether the provided document matches the registered version.

Claim 14. The system of Claim 1, further comprising a health scoring module configured to compute a composite numerical health score for the governed community by: computing a dues collection component score based on a ratio of current-period paying members to total active property token holders; computing a treasury health component score based on a ratio of the reserve fund balance to a target reserve fund allocation; computing a governance participation component score based on an average voter turnout across recent governance proposals; computing a document compliance component score based on a presence of required document categories in a document registry; computing a community size component score based on a count of active property tokens relative to a scaling target; and summing the component scores, each multiplied by a predefined weighting coefficient, to produce the composite health score in a range of zero to one hundred.

Claim 15. The system of Claim 14, wherein the weighting coefficients are: thirty percent (30%) for the dues collection component; twenty-five percent (25%) for the treasury health component; twenty percent (20%) for the governance participation component; fifteen percent (15%) for the document compliance component; and ten percent (10%) for the community size component.

Claim 16. The system of Claim 1, wherein all voting operations, proposal submissions, treasury transactions, and document registrations are recorded as immutable transaction log entries on the blockchain network, creating a tamper-evident audit trail accessible to any party.

Claim 17. The system of Claim 1, wherein the governance contract further comprises an enforcement mechanism that prevents any individual address from accumulating voting weight exceeding one unit per represented property, and wherein voting weight may be delegated to another address but the delegated weight is bounded by the number of soulbound property tokens held by the delegating address.

Claim 18. A computer-implemented method for tokenized homeowners association governance, the method comprising: minting, by a smart contract executing on a blockchain, a non-fungible property token associated with a residential lot, the property token encoding property metadata on-chain and carrying one unit of voting weight; restricting, by the smart contract, transfer of the property token such that the token may only be transferred upon authorization by an address holding a designated transfer approval role; receiving, by a governance smart contract, a governance proposal associated with a proposal category selected from a plurality of predefined categories; applying, by the governance smart contract, a quorum threshold and an approval threshold corresponding to the assigned proposal category to evaluate whether the proposal succeeds; and automatically allocating, by a treasury smart contract, an incoming dues payment between an operating fund balance and a reserve fund balance according to a predefined split ratio without requiring manual intervention.

Claim 19. The method of Claim 18, further comprising: computing a cryptographic hash of a governance document; storing the cryptographic hash in a document registry smart contract on the blockchain; storing the document content at a permanent decentralized storage address; and verifying, in response to a user-provided file, whether the file's computed hash matches a hash stored in the document registry.

Claim 20. The method of Claim 18, further comprising: reading, from one or more smart contracts on the blockchain, current values of dues collection rate, treasury reserve ratio, governance proposal participation rate, document category coverage, and active property token count; and computing, from the read values and predefined weighting coefficients, a composite community health score bounded between zero and one hundred.

Claim 21. A non-transitory computer-readable medium storing smart contract bytecode that, when executed by a blockchain virtual machine, implements: a property token registry maintaining a one-to-one mapping between residential lot identifiers and non-fungible token identifiers, wherein each token carries one unit of voting weight and wherein transfers of tokens between non-zero addresses are reverted unless the transaction originates from an authorized transfer approver address; a governance proposal system that associates each submitted proposal with a category and enforces category-specific minimum participation thresholds and minimum approval thresholds independently of other proposal categories; and a treasury allocation system that, upon receipt of a stablecoin payment, immediately and automatically partitions the payment between an operating fund accounting variable and a reserve fund accounting variable at a predetermined ratio.

Claim 22. The non-transitory computer-readable medium of Claim 21, wherein the property token registry further maintains on-chain structured data for each token comprising at least a lot number

field of unsigned integer type, a street address field of string type, and a square footage field of unsigned integer type.

Claim 23. The non-transitory computer-readable medium of Claim 21, wherein the governance proposal system further implements a timelock delay between proposal passage and proposal execution, the timelock delay being a function of the proposal's assigned category, with higher-impact categories associated with longer timelock delays.

Claim 24. The non-transitory computer-readable medium of Claim 21, wherein expenditures from the treasury are executable only upon receipt of an execution call from a timelock controller contract whose execution is gated by successful completion of a governance vote, such that no human actor may unilaterally withdraw funds from the treasury.

ABSTRACT

A blockchain-based homeowners association (HOA) governance platform deployed on a Layer 2 Ethereum-compatible network comprises five integrated inventive components. First, a soulbound property NFT system mints ERC-721 tokens representing individual residential lots, wherein each token carries one unit of voting weight, transfers require role-based board approval preventing unauthorized accumulation, and property metadata is stored on-chain. Second, a tiered governance system classifies proposals into Routine (15% quorum), Financial (33% quorum), Governance (51% quorum), and Constitutional (67% quorum, 66.7% approval) categories, enforcing category-specific participation and approval thresholds at the smart contract level with category-variable timelock delays. Third, an automated treasury management system receives HOA dues in USDC stablecoin and immediately allocates payments at an 80%/20% operating/reserve split, with all expenditures gated by governance execution, creating complete financial transparency and eliminating embezzlement risk. Fourth, an immutable document registry stores cryptographic content hashes of HOA documents on-chain while referencing full document content stored on the Arweave permanent decentralized storage network, with a browser-side drag-and-drop verification interface enabling any party to instantly verify document authenticity. Fifth, a composite community health scoring algorithm computes a real-time score from zero to one hundred by weighting five on-chain metrics: dues collection rate (30%), treasury health (25%), governance participation (20%), document compliance (15%), and community size (10%), providing a single objective metric for community governance quality.

This provisional patent application is filed for the purpose of establishing a priority date. It is not examined and does not by itself result in the grant of a patent. A corresponding non-provisional application must be filed within twelve (12) months of this provisional application's filing date to claim the benefit of this filing date.

Inventors: [To be identified in non-provisional application]
Filing Date: [Date of filing]
Docket No.: SUVREN-HOA-001-PROV

DISCLAIMER: This document is a draft provisional patent application prepared for internal review purposes. It should be reviewed and filed by a licensed patent attorney or patent agent. The claims and disclosure herein reflect the inventors' understanding of the inventions and should be professionally evaluated for patentability, prior art clearance, and claim scope before filing.